

Types of variable

- We must *declare* the *type* of every variable we use in C.
- Every variable has a *type* (e.g. `int`) and a *name*.
- This prevents some bugs caused by spelling errors (misspelling variable names).
- Declarations of types should always be together at the top of main or a function (see later).
- Other types are `char`, `signed`, `unsigned`, `long`, `short` and `const`.

Identifiers and Keywords

- **Identifiers**

- Names given to various program elements (variables, constants, functions, etc.)
- May consist of *letters*, *digits* and the *underscore* ('_') character, with no space between.
- First character must be a letter or underscore.
- An identifier can be arbitrary long.
 - Some C compilers recognize only the first few characters of the name (16 or 31).
- **Case sensitive**
 - 'area', 'AREA' and 'Area' are all different.

Valid and Invalid Identifiers

- Valid identifiers

X

abc

simple_interest

a123

LIST

stud_name

Empl_1

Empl_2

avg_empl_salary

- Invalid identifiers

10abc

my-name

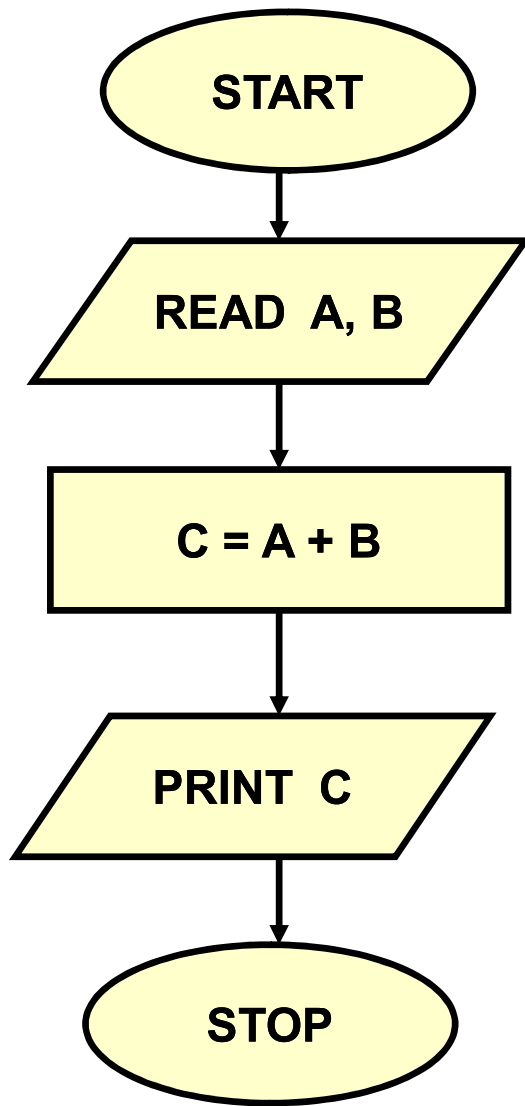
“hello”

simple interest

(area)

%rate

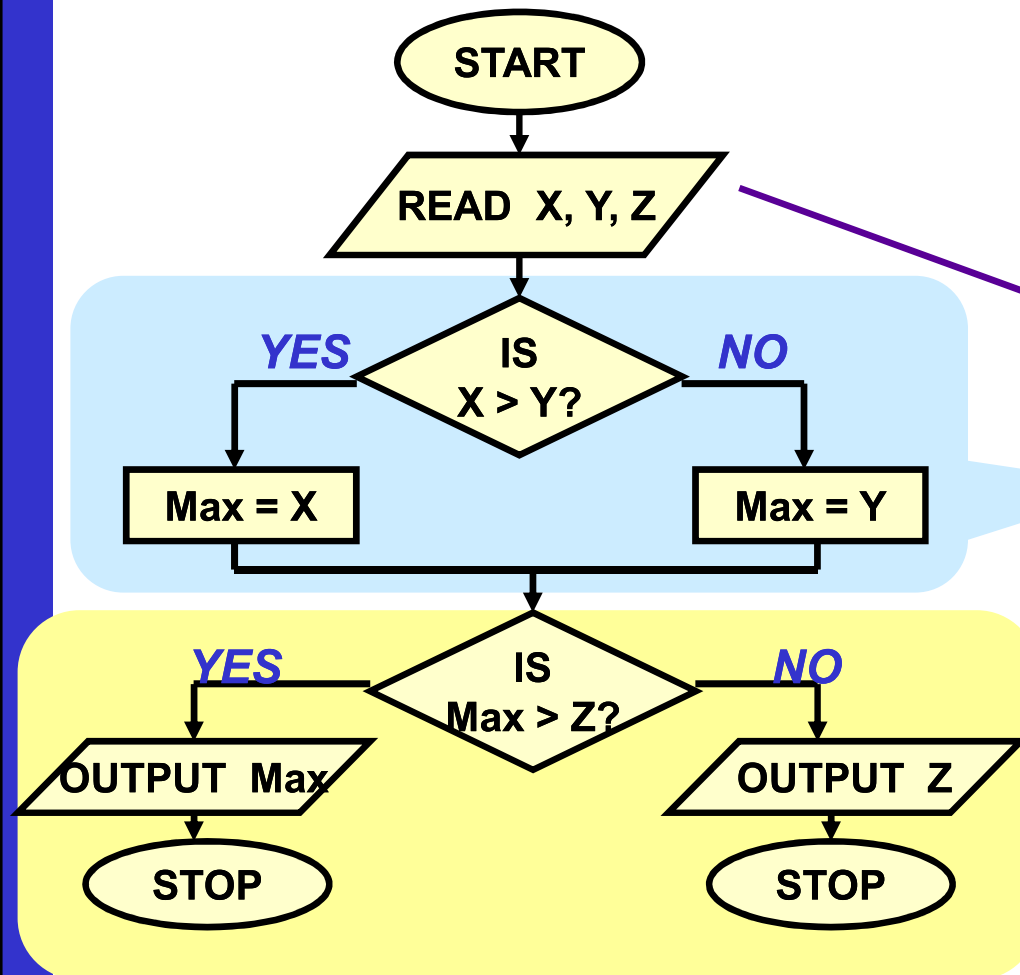
Another Example: *Adding two numbers*



```
#include <stdio.h>
main()
{
    int a, b, c;
    scanf ("%d%d", &a, &b) ;
    c = a + b;
    printf ("%d", c) ;
}
```

Variable Declaration

Example: *Largest of three numbers*



```
#include <stdio.h>
/* FIND THE LARGEST OF THREE NUMBERS */
main()
{
    int a, b, c, max;
    scanf ("%d %d %d", &x, &y, &z);

    if (x>y)
        max = x;
    else max = y;

    if (max > z)
        printf("Largest is %d", max);
    else printf("Largest is %d", z);
}
```

Largest of three numbers: *Another way*

```
#include <stdio.h>

/* FIND THE LARGEST OF THREE NUMBERS */

main()
{
    int  a, b, c;
    scanf ("%d %d %d", &a, &b, &c);
    if ((a>b) && (a>c))  /* Composite condition check */
        printf ("\n Largest is %d", a);
    else
        if (b>c)        /* Simple condition check */
            printf ("\n Largest is %d", b);
        else
            printf ("\n Largest is %d", c);
}
```

Use of functions: *Area of a circle*

```
#include <stdio.h>
```

```
#define PI 3.1415926
```

Macro definition

```
/* Function to compute the area of a circle */
```

```
float myfunc (float r)
```

```
{    float a;
```

```
    a = PI * r * r;
```

```
    return (a);    /* return result */
```

```
}
```

Function definition

Function argument

```
main()
```

```
{    float radius, area;
```

```
    float myfunc (float radius);
```

Function declaration
(return value defines the type)

```
    scanf ("%f", &radius);
```

```
    area = myfunc (radius);
```

Function call

```
    printf ("\n Area is %f \n", area);
```

```
}
```

Structure of a C program

- Every C program consists of one or more functions.
 - One of the functions must be called *main*.
 - The program will always begin by executing the main function.
- Each function must contain:
 - A function *heading*, which consists of the function *name*, followed by an optional list of *arguments* enclosed in parentheses.
 - A list of argument *declarations*.
 - A *compound statement*, which comprises the remainder of the function.

Desirable Programming Style

- **Clarity**
 - The program should be clearly written.
 - It should be easy to follow the program logic.
- **Meaningful variable names**
 - Make variable/constant names meaningful to enhance program clarity.
 - 'area' instead of 'a'
 - 'radius' instead of 'r'
- **Program documentation**
 - Insert comments in the program to make it easy to understand.
 - Never use too many comments.
- **Program indentation**
 - Use proper indentation.
 - Structure of the program should be immediately visible.

Indentation Example: *Good Style*

```
#include <stdio.h>
```

```
/* FIND THE LARGEST OF THREE NUMBERS */
```

```
main()
```

```
{
```

```
    int  a, b, c;
```

```
    scanf("%d%d%d", &a, &b, &c);
```

```
    if ((a>b) && (a>c))
```

```
        printf("\n Largest is %d", a);
```

```
    else
```

```
        if (b>c)
```

```
            printf("\n Largest is %d", b);
```

```
        else
```

```
            printf("\n Largest is %d", c);
```

```
}
```

Indentation Example: *Bad Style*

```
#include <stdio.h>

/* FIND THE LARGEST OF THREE NUMBERS */
main()
{
int  a, b, c;
scanf("%d%d%d", &a, &b, &c);
if ((a>b) && (a>c))
printf("\n Largest is %d", a);
    else
if (b>c)
    printf("\n Largest is %d", b);
else
printf("\n Largest is %d", c);
}
```

Data Types in C

int :: integer quantity

Typically occupies 4 bytes (32 bits) in memory.

char :: single character

Typically occupies 1 byte (8 bits) in memory.

float :: floating-point number (a number with a decimal point)

Typically occupies 4 bytes (32 bits) in memory.

double :: double-precision floating-point number

Contd.

- Some of the basic data types can be augmented by using certain data type qualifiers:
 - short
 - long
 - signed
 - unsigned
- Typical examples:
 - short int
 - long int
 - unsigned int

Some Examples of Data Types

- **int**

0, 25, -156, 12345, -99820

- **char**

'a', 'A', '*', '/', ''

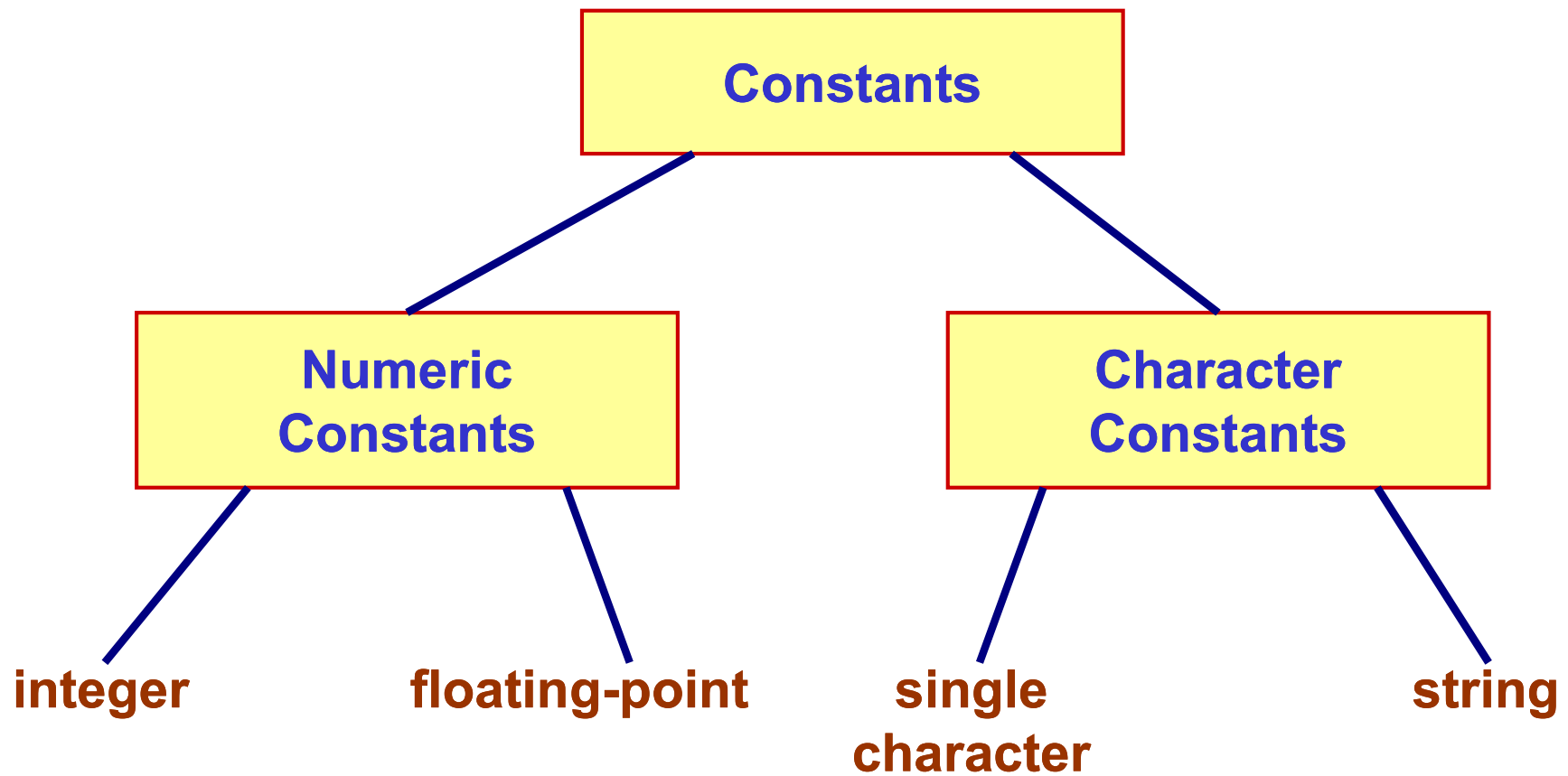
- **float**

23.54, -0.00345, 25.0

2.5E12, 1.234e-5

E or e means “10 to the power of”

Constants



Integer Constants

- **Consists of a sequence of digits, with possibly a plus or a minus sign before it.**
 - **Embedded spaces, commas and non-digit characters are not permitted between digits.**
- **Maximum and minimum values (for 32-bit representations)**
 - Maximum :: 2147483647**
 - Minimum :: – 2147483648**

Floating-point Constants

- Can contain fractional parts.
- Very large or very small numbers can be represented.
23000000 can be represented as 2.3e7
- Two different notations:
 1. **Decimal notation**
25.0, 0.0034, .84, -2.234
 2. **Exponential (scientific) notation**
3.45e23, 0.123e-12, 123E2

e means “10 to the power of”

Single Character Constants

- Contains a single character enclosed within a pair of single quote marks.
 - Examples :: '2', '+', 'Z'
- Some special backslash characters
 - '\n' new line
 - '\t' horizontal tab
 - '\'' single quote
 - '\"' double quote
 - '\\' backslash
 - '\0' null

String Constants

- **Sequence of characters enclosed in double quotes.**
 - **The characters may be letters, numbers, special characters and blank spaces.**
- **Examples:**
 - “nice”, “Good Morning”, “3+6”, “3”, “C”**
- **Differences from character constants:**
 - **‘C’ and “C” are not equivalent.**
 - **‘C’ has an equivalent integer value while “C” does not.**

Declaration of Variables

- **There are two purposes:**
 1. It tells the compiler what the variable name is.
 2. It specifies what type of data the variable will hold.
- **General syntax:**
data-type variable-list;
- **Examples:**
`int velocity, distance;`
`int a, b, c, d;`
`float temp;`
`char flag, option;`